

OWT-Krylov Manual

A distributed Krylov solver library for unstructured grids

Complete technical reference: core data structures, execution and low-level kernels, the Krylov solver API, preconditioners, coarse levels and multigrid, MPI and PETSc integration, and the SpecWave compatibility layer.

OWT-Krylov Project

2026-08-31

Companion documents: `README.md`, `docs/ROADMAP.md`, `docs/SPECWAVE_PORT_MATRIX.md`,
`docs/LIMON_BENCHMARK.md`, `literature/REPOSITORY_REVIEW.md`.

Contents

About this manual	3
1 Design and scope	4
2 Requirements and building	5
2.1 CMake options	5
2.2 Build recipes	5
2.3 Install and consume	6
3 Core data structures	7
3.1 The layout invariant	7
3.2 BlockVector<T>	7
3.3 DistributedLayout	8
3.4 Matrix storage: three forms	8
3.5 Composite operators	10
4 Execution and low-level kernels	11
4.1 SIMD kernels	11
4.2 Execution policies	11
4.3 Node orderings	12
4.4 Reduction policies	13
5 Solving linear systems: the Krylov API	15
5.1 The common contract	15
5.2 SolverOptions<T>	15
5.3 SolverResult<T>, SolverStatus, BreakdownReason	16
5.4 GMRES and FGMRES	17
5.5 The BiCGSTAB family	18
5.6 IDR(s)	19
5.7 Recycling and GCRO-DR	20
5.8 Reusable workspaces	21
6 Preconditioners	22
6.1 Identity and Jacobi	22
6.2 Local SSOR	22
6.3 ILU(0)	23
6.4 Restricted Additive Schwarz	23
6.5 Stationary methods and smoothers	25
7 Coarse levels and multigrid	27
7.1 SubdomainConstantCoarseCorrection	27
7.2 MultiplicativeTwoLevelPreconditioner	27
7.3 AggregationMultigrid	28
7.4 PetscSubdomainCoarseCorrection	28

8	Distributed execution (MPI)	29
8.1	MpiHaloExchange<T>	29
8.2	OverlappedDistributedBlockOperator	29
8.3	Overlap discovery for RAS	30
8.4	Choosing a reduction policy	30
9	PETSc adapter	31
9.1	PetscBlockCsrSolver	31
9.2	PetscSubdomainCoarseCorrection	32
10	SpecWave compatibility layer	33
10.1	SpecWaveSolver enum and dispatcher	33
10.2	ID-to-implementation table	34
11	Complete examples	37
11.1	Serial GMRES	37
11.2	MPI-distributed GMRES	37
11.3	Restricted Additive Schwarz (depth-1)	38
11.4	PETSc adapter with independent residual check	39
11.5	GCRO-DR recycling across related solves	40
12	Benchmarking	41
12.1	Unstructured development benchmark	41
12.2	SpecWave Limon protocol	41
13	Testing	42
14	Known rough edges	43
15	Claim boundaries	45
16	Header index	46

About this manual

This is the complete technical reference for OWT-Krylov: a C++20, header-only, MPI-optional distributed Krylov solver library for unstructured grids. It assumes the reader is comfortable with sparse linear algebra and Krylov methods; it does not re-derive the algorithms, only documents precisely what this implementation does, how the pieces compose, and where the sharp edges are.

For a one-page feature list see `README.md`. For the SpecWave legacy solver-ID mapping see `docs/SPECWAVE_PORT_MATRIX.md`. For the cross-repository design review that shaped this API see `literature/REPOSITORY_REVIEW.md`.

1 Design and scope

OWT-Krylov is the standalone extraction of SpecWave’s distributed solver architecture for unstructured grids. Its storage unit is a contiguous block per unstructured-grid node — owned nodes first, then ghost nodes, each block `block_size()` scalars wide — not scalar CSR. Every matrix, vector, and preconditioner type in the library is built around that layout.

The library occupies a specific niche relative to full solver frameworks (PETSc, Trilinos, hypre, Ginkgo). It is:

- a compact C++20 header-only library, not a package ecosystem;
- domain decomposition and unstructured node ownership as first-class public API concepts (`DistributedLayout`, owned/ghost `BlockVector`), not something bolted on;
- serial and MPI execution through the same algorithmic interface — every solver is templated on an `Operator/Preconditioner/Reduction` policy, so swapping `SerialReduction<T>` for `MpiReduction<T>` is the only change needed to go distributed;
- the complete, inspectable SpecWave nonsymmetric solver family (GMRES, BiCGSTAB in three communication profiles, IDR(s)) plus domain-decomposition preconditioners, all native C++, no virtual dispatch on the hot path;
- adapters to PETSc rather than in-tree reimplementations of algebraic multigrid — PETSc is used as both an optional backend and a correctness oracle, never reproduced.

OWT does not initialize or finalize MPI or PETSc. Runtime ownership of both stays with the application. The library also does not own mesh geometry: it consumes ownership/adjacency products of a partitioner (`DistributedLayout`) and accepts matrix-free operators, so application-specific physics (boundary conditions, refraction terms, spectral coupling, NML parsing in SpecWave’s case) never needs to be flattened into the block-CSR core.

2 Requirements and building

- C++20 compiler (the library uses `std::floating_point`/`std::integral` concepts, `std::span`, and designated aggregate patterns throughout).
- CMake ≥ 3.20 .
- Optional: an MPI implementation with a C++ compiler wrapper, PETSc (built against the *same* MPI implementation as the compiler/MPI target), LAPACK, and an OpenMP-Target-capable compiler.

2.1 CMake options

Option	Default	Effect
OWT_KRYLOV_ENABLE_MPI	OFF	Links <code>MPI::MPI_CXX</code> ; defines <code>OWT_KRYLOV_ENABLE_MPI=1</code> , which gates <code>mpi_halo.hpp</code> , the MPI reduction policies in <code>reduction.hpp</code> , and everything in <code>overlap.hpp/overlap_plan.hpp/coarse.hpp</code> .
OWT_KRYLOV_ENABLE_PETSC	OFF	Requires <code>OWT_KRYLOV_ENABLE_MPI=ON</code> (fatal CMake error otherwise). Links PETSc via <code>pkg_check_modules</code> ; defines <code>OWT_KRYLOV_ENABLE_PETSC=1</code> , gating <code>petsc_adapter.hpp</code> and <code>petsc_coarse.hpp</code> .
OWT_KRYLOV_ENABLE_LAPACK	OFF	Links <code>LAPACK::LAPACK</code> ; defines <code>OWT_KRYLOV_ENABLE_LAPACK=1</code> , gating <code>gcrodr.hpp</code> (harmonic-Ritz GCRO-DR extraction).
OWT_KRYLOV_ENABLE_OPENMP_TARGET	OFF	Links <code>OpenMP::OpenMP_CXX</code> ; defines <code>OWT_KRYLOV_ENABLE_OPENMP_TARGET=1</code> , gating <code>OpenMPTargetExecutionPolicy</code> in <code>execution.hpp</code> .
OWT_KRYLOV_BUILD_TESTS	ON	Builds the <code>tests/</code> suite via CTest.
OWT_KRYLOV_BUILD_BENCHMARKS	OFF	Builds <code>owt_krylov_unstructured_benchmark</code> and, if MPI is enabled, <code>owt_krylov_distributed_benchmark</code> .

`owt_krylov` is an INTERFACE library target (header-only); the CMake project only compiles anything when tests or benchmarks are enabled.

2.2 Build recipes

Serial:

```
cmake -S . -B build
cmake --build build
ctest --test-dir build --output-on-failure
```

MPI:

```
cmake -S . -B build-mpi -DOWT_KRYLOV_ENABLE_MPI=ON
cmake --build build-mpi
ctest --test-dir build-mpi --output-on-failure
```

PETSc (must use the same MPI implementation as the compiler and MPI target):

```
cmake -S . -B build-petsc \
-DOWT_KRYLOV_ENABLE_MPI=ON \
-DOWT_KRYLOV_ENABLE_PETSC=ON
```

GCRO-DR and OpenMP Target are explicit optional features:

```
cmake -S . -B build-advanced \
-DOWT_KRYLOV_ENABLE_LAPACK=ON \
-DOWT_KRYLOV_ENABLE_OPENMP_TARGET=ON
```

2.3 Install and consume

```
cmake --install build --prefix /desired/prefix
```

```
find_package(OWTKrylov CONFIG REQUIRED)
target_link_libraries(your_target PRIVATE OWT::Krylov)
```

Including everything:

```
#include <owt/krylov/owt_krylov.hpp>
```

`owt_krylov.hpp` aggregates every header listed in §16. Individual headers can be included directly for a narrower dependency footprint (e.g. `specwave_compat.hpp` alone does *not* pull in `krylov_solvers.hpp`, so a translation unit that only wants `solve_specwave_native` still needs to include `krylov_solvers.hpp` itself if it also calls `gmres/bicgstab` directly).

3 Core data structures

Everything in this chapter lives in `include/owt/krylov/core.hpp`, `block_csr.hpp`, `dense_block_csr.hpp`, `split_block_csr.hpp`, and `distributed_layout.hpp`.

3.1 The layout invariant

Every distributed container in OWT-Krylov follows one rule, stated in `core.hpp`'s own comment on `BlockVector`: **owned nodes first, ghost nodes second, and a contiguous block per node**. For a vector with `owned_nodes()` owned and `ghost_nodes()` ghost nodes and `block_size()` components per node:

- scalars `[0, owned_size())` are the owned region (`owned_size() = owned_nodes() * block_size()`);
- scalars `[owned_size(), local_size())` are the ghost region.

Every BLAS-1 free function (`axpy`, `scale`, `copy_owned`, `assign_linear_combination`) and every dot/norm reduction operates on the **owned region only** — ghosts are never summed into a reduction, since each value is owned by exactly one rank globally. Halo/ghost updates are always a separate, explicit step (a Halo policy's exchange, or `begin/end`), never implicit inside a BLAS-1 call.

3.2 `BlockVector<T>`

`core.hpp`. `T` is `std::floating_point`.

```
BlockVector(); // empty
BlockVector(std::size_t owned_nodes, std::size_t ghost_nodes,
            std::size_t block_size, T initial_value = T(0)); // owning
static BlockVector view(std::size_t owned_nodes, std::size_t ghost_nodes,
                       std::size_t block_size, std::span<T> values); // non-owning
```

`owned_nodes == 0` and `block_size == 0` are rejected (`std::invalid_argument`); `ghost_nodes == 0` is allowed. `view()` requires `values.size() == (owned_nodes+ghost_nodes)*block_size` exactly.

Copying an **owning** vector deep-copies; copying a **view** produces another view of the same underlying storage. The caller of `view()` must keep the wrapped storage alive and at a fixed address for the view's lifetime.

Key accessors: `owned_nodes()`, `ghost_nodes()`, `local_nodes()`, `block_size()`, `owned_size()`, `local_size()`, `owns_memory()`, `data()`, `owned()/ghosts()` (spans), `node(local_index)` (span of one node's block, throws `std::out_of_range` if `local_index >= local_nodes()`), `fill_owned`, `fill`, `clone_layout()` (new owning vector, same shape, independent storage), `same_layout(other)` (compares `owned_nodes/ghost_nodes/block_size` only).

Free functions (all throw `std::invalid_argument` on a layout mismatch, all owned-region-only):

```
void copy_owned(const BlockVector<T>& source, BlockVector<T>& destination);
void axpy(T alpha, const BlockVector<T>& x, BlockVector<T>& y); // y += alpha*x
```

```

void scale(T alpha, BlockVector<T>& x); // x *= alpha
void assign_linear_combination(BlockVector<T>& out, T alpha,
                               const BlockVector<T>& x, T beta,
                               const BlockVector<T>& y); // out =
                               alpha*x + beta*y

```

No thread-safety mechanism exists anywhere in this type; nothing here is safe for concurrent mutation.

3.3 DistributedLayout

`distributed_layout.hpp`. Not templated on a scalar type — pure bookkeeping supplied by an unstructured mesh partitioner, describing node ownership. It has no code-level coupling to `BlockVector/BlockCsrMatrix` — the caller is responsible for keeping a `DistributedLayout`'s node counts consistent with whatever `BlockVector`/matrix it is used alongside.

```

using global_index_type = std::uint64_t;
DistributedLayout(global_index_type global_nodes, std::size_t block_size,
                  std::vector<global_index_type> owned_global_nodes,
                  std::vector<global_index_type> ghost_global_nodes,
                  std::vector<int> ghost_owners,
                  std::vector<global_index_type> algebraic_local_to_global_nodes);

```

Immutable after construction (every accessor `const`, no setters). Validated at construction: `global_nodes = 0!`, `block_size = 0!`, `owned_global_nodes` non-empty; `ghost_global_nodes.size() == ghost_owners.size()`; `algebraic_local_to_global_nodes.size() == owned_global_nodes.size() + ghost_global_nodes.size()`; every `owned/ghost global id < global_nodes` and unique across `owned+ghost` together; every `algebraic_local_to_global_nodes` entry `< global_nodes` and unique *within that array* (checked with a separate uniqueness set — see §14 for what this does **not** cross-check).

Accessors: `global_nodes()`, `block_size()`, `owned_nodes()`, `ghost_nodes()`, `owned_global_nodes()`, `ghost_global_nodes()`, `ghost_owners()` (parallel array to `ghost_global_nodes()`, the owning rank of each ghost — meaning otherwise opaque to this header), `algebraic_local_to_global_nodes()`.

3.4 Matrix storage: three forms

All three share the same layout invariant (owned rows, ghost columns permitted) and the same `apply/apply_rows` contract, but store block couplings differently. Pick based on whether a node's `block_size()` components couple to each other, and whether the application already owns the numeric arrays in a particular layout.

Type	Per-entry storage	Owning or view	SIMD path
<code>BlockCsrMatrix<T, Index></code>	<code>block_size</code> diagonal scalars (component-diagonal — no cross-component coupling within an entry)	Both (owning ctor and <code>view()</code>)	<code>simd::fused_multiply_add</code>

Type	Per-entry storage	Owning or view	SIMD path
<code>DenseBlockCsrMatrix<T, Index></code>	full block_size x block_size row-major dense block	Both	Plain nested loops — no SIMD; $O(\text{block_size}^2)$ per entry vs. $O(\text{block_size})$ for <code>BlockCsrMatrix</code>
<code>SplitBlockCsrMatrixView<T, Index></code>	diagonal array [owned node][component] + separate off-diagonal CSR [edge][component]	View-only, always non-owning; diagonal /off_diagonal are <code>const</code> spans (mutated only by the application directly)	<code>simd::</code> <code>fused_multiply_add</code>

`BlockCsrMatrix` is the general-purpose type (used throughout the examples in §11). `DenseBlockCsrMatrix` is for genuinely block-coupled DOFs (e.g. tightly-coupled multi-physics unknowns at a node). `SplitBlockCsrMatrixView` exists specifically to borrow `SpecWave`’s native array layout — separate diagonal and edge-coefficient arrays — with the structural arrays (row offsets, columns) cached once while both numeric arrays continue to point directly at application-owned storage; mutating the application’s arrays is immediately visible on the next `apply()`, no re-registration needed.

All three throw `std::invalid_argument` from their constructor/view() on a structural violation (missing/mismatched-size row offsets, non-monotone offsets, an out-of-range column index) and from every `apply/apply_rows` on a vector/matrix layout mismatch (`owned_nodes/ghost_nodes/block_size` must agree exactly). `BlockCsrMatrix::diagonal()` throws if a row has no self-loop entry; `SplitBlockCsrMatrixView::diagonal()` cannot fail this way because the diagonal is stored directly, not searched for.

Every matrix precomputes `interior_rows()` (owned rows whose entries never touch a ghost column) and `boundary_rows()` (owned rows with at least one ghost-column entry) at construction — the partition used throughout the library to overlap communication with computation (see §4.2 and the asynchronous Gauss–Seidel discussion in §6.5).

Minimal construction

```
BlockCsrMatrix<double> matrix(
    /*owned_nodes=*/3, /*ghost_nodes=*/0, /*block_size=*/2,
    /*row_offsets=*/ {0, 2, 5, 7},
    /*column_indices=*/{0, 1, 0, 1, 2, 1, 2},
    /*values=*/      { /* block_size scalars per entry, in entry order */});
```

```
row_offsets.size() == owned_nodes+1, column_indices.size() == row_offsets.back(), values
.size() == column_indices.size()*block_size (component-diagonal form) or column_indices.
```

`size()*block_size*block_size (DenseBlockCsrMatrix, row-major per entry).`

3.5 Composite operators

`block_csr.hpp` also defines:

```
struct NoHaloExchange { /* every method is a no-op */ };

template<class Matrix, class Halo = NoHaloExchange>
class DistributedBlockOperator {
    DistributedBlockOperator(Matrix matrix, Halo halo = {});
    template<std::floating_point T> void apply(BlockVector<T>&, BlockVector<T>&);
};
```

`apply` calls `halo_.exchange(input)` **then** `matrix_.apply(input, output)`. With the default `NoHaloExchange`, this performs **no ghost update** — a genuinely distributed use requires supplying a real `Halo` (see `MpiHaloExchange` in §8). A deduction guide lets `DistributedBlockOperator(matrix)` bind the matrix by reference (not copy) when given an lvalue.

`mpi_halo.hpp`'s `OverlappedDistributedBlockOperator<Matrix, Halo>` is the communication/computation-overlapping counterpart — see §8.2.

4 Execution and low-level kernels

4.1 SIMD kernels

`simd.hpp` (`owt::krylov::simd`). Two free functions, both `noexcept`, no bounds checking:

```
template<std::floating_point T>
void fused_multiply_add(T* out, const T* coefficient, const T* input, std::size_t size
);
// out[i] += coefficient[i] * input[i]

template<std::floating_point T>
void axpy(T* out, T alpha, const T* input, std::size_t size);
// out[i] += alpha * input[i]
```

Dispatch is **entirely compile-time**, in priority order `__AVX512F__` → `__AVX2__` → `__ARM_NEON` / `__ARM_NEON__` → scalar remainder loop, each with a nested `__FMA__` check to prefer true fused multiply-add over separate multiply+add. These are standard compiler-predefined ISA macros (set via `-march=`/`-mavx2`/`-mfma`/target flags), **not** OWT CMake options — nothing in `CMakeLists.txt` sets architecture flags automatically. A binary built with `-mavx2` will SIGILL on a CPU lacking AVX2; a binary built with none of these flags always uses the scalar path regardless of the actual CPU. 32-bit ARM has no NEON double-precision path (falls through to scalar for double); AArch64 does.

Practical note: `fused_multiply_add` is called once per CSR entry with `size = block_size()`. For small block sizes (3–9, typical for PDE systems) the vector width (4/8/16 lanes) may exceed `size`, so the vectorized branch never executes and the whole call falls through to the scalar tail loop — don't expect SIMD speedup here for small blocks. `axpy` (called with `size = owned_size()`, i.e. the whole vector) is where SIMD actually pays off.

4.2 Execution policies

`execution.hpp`. A policy selects *how* the local SpMV of a block-CSR operator runs.

```
struct HostExecutionPolicy {
    template<class Matrix, std::floating_point T>
    void apply(const Matrix&, const BlockVector<T>&, BlockVector<T>&) const; //
    forwards to matrix.apply
    static constexpr const char* name() noexcept; // "host"
    static constexpr bool accelerator_available() noexcept; // always false
};

#ifdef OWT_KRYLOV_ENABLE_OPENMP_TARGET
struct OpenMPTargetExecutionPolicy {
    static constexpr const char* name() noexcept; // "openmp-target"
    static bool accelerator_available() noexcept; //
    omp_get_num_devices() > 0
    // apply(BlockCsrMatrix<T, Index>, ...) and apply(SplitBlockCsrMatrixView<T, Index>,
    ...)
    // reimplement the SpMV kernel with 'omp target teams distribute parallel for
    collapse(2)'
};
```

```
// over (row, component) pairs -- does NOT call matrix.apply().
};
#endif
```

OpenMPTargetExecutionPolicy supports exactly two matrix types (BlockCsrMatrix, SplitBlockCsrMatrixView) — it is not generic like HostExecutionPolicy. **Every call re-transfers the entire matrix and input vector to the device and copies the entire output back** — there is no persistent device-resident state (target enter data) in this header. Budget the transfer cost per apply() call; the README’s claim boundary is explicit that no GPU speed claim is made without device-resident workload measurements.

```
template<class Matrix, class ExecutionPolicy = HostExecutionPolicy, class Halo =
    NoHaloExchange>
class PolicyBlockOperator {
    PolicyBlockOperator(Matrix& matrix, ExecutionPolicy = {}, Halo = {});
    template<std::floating_point T> void apply(BlockVector<T>& input, BlockVector<T>&
        output);
    ExecutionPolicy& execution_policy() noexcept;
};
```

Holds a raw pointer to matrix (caller keeps it alive); apply always runs halo_.exchange(input) before the local kernel (a no-op under the default NoHaloExchange).

4.3 Node orderings

ordering.hpp. Four algorithms plus an inverse-permutation helper, all operating on the **local (owned-only) subgraph** — ghost/off-rank columns are dropped before any of these run, so reordering is per-rank, never global.

```
enum class NodeOrdering { identity, hilbert, reverse_cuthill_mckee,
    approximate_minimum_degree, nested_dissection };

std::vector<std::size_t> reverse_cuthill_mckee_order(const BlockCsrMatrix<T, Index>&);
std::vector<std::size_t> approximate_minimum_degree_order(const BlockCsrMatrix<T, Index>
    &);
std::vector<std::size_t> nested_dissection_order(const BlockCsrMatrix<T, Index>&);
std::vector<std::size_t> hilbert_order(std::span<const std::pair<double, double>>
    coordinates);
std::vector<std::size_t> inverse_permutation(std::span<const std::size_t> new_to_old);
```

Every ordering function returns a **new-to-old** permutation (position = new index, value = old node index); inverse_permutation converts to **old-to-new** and validates the input is a genuine permutation (throws std::invalid_argument on an out-of-range or duplicate entry).

- RCM and Hilbert target cache/bandwidth locality (RCM via graph BFS distance, Hilbert via geometric proximity — hilbert_order needs externally supplied 2D coordinates, matching the matrix’s local node indexing; there is no 3D variant).
- approximate_minimum_degree_order and nested_dissection_order target fill-in reduction for a factorization-based preconditioner. **approximate_minimum_degree_order is not the AMD algorithm** despite the name — it is a plain greedy minimum-degree ordering with an $O(n)$ linear pivot scan per elimination step ($O(n^2)$ total), not AMD’s near-linear approximate-degree/quotient-graph structure. Don’t expect AMD-library scalability from it on large local graphs.

- `nested_dissection_order` recurses down to a leaf threshold of 8 nodes, falling back to a non-separating positional midpoint split (empty separator) whenever a subgraph’s BFS partition is degenerate (e.g. disconnected) — this weakens but doesn’t break the ordering’s fill-reducing property in that case.

No function anywhere in the library consumes a `NodeOrdering` value. The enum is a labeling vocabulary for callers to select which of the four standalone functions to call; there is no `order(matrix, NodeOrdering::x)` dispatcher, and no solver or preconditioner in this repository applies a reordering automatically. If you want a reordered solve, you must permute the matrix/vectors yourself before constructing them and keep any halo metadata in sync — this header provides no such synchronization.

4.4 Reduction policies

`reduction.hpp`. A reduction policy computes dot products, norms, and batched sums; every Krylov solver is templated on `Reduction` (default `SerialReduction<T>`) and calls `dot/norm/sum/begin_sum/end` uniformly regardless of which policy is chosen. **This is the single knob that turns a solver from single-rank to MPI-distributed** — no solver code changes, only the policy object passed in.

Policy	Scope	Accumulation	Determinism
<code>SerialReduction<T></code>	Single rank	T, Kahan-compensated	Deterministic (fixed order)
<code>MixedPrecisionSerialReduction<T></code>	Single rank	long double, Kahan-compensated, narrowed to T at the end	Deterministic
<code>MpiReduction<T></code>	MPI, float/double only	<code>MPI_Allreduce</code> / <code>MPI_Iallreduce</code> , <code>MPI_SUM</code>	Not bit-reproducible run-to-run (implementation-defined reduction tree order)
<code>MpiDeterministicReduction<T></code>	MPI	Gather to rank 0, sum in strict rank order (Kahan), broadcast	Reproducible for a fixed rank count and rank/data mapping ; not partition-independent. <code>begin_sum</code> is blocking despite the name — don’t use it to measure pipelined-algorithm performance. Verification tool, not a production path.
<code>MpiMixedPrecisionReduction<T></code>	MPI, float/double only	Local dot in long double (via <code>MixedPrecisionSerialReduction</code>) , then <code>MPI_Allreduce</code> in double, narrowed to T	Same non-reproducibility caveat as <code>MpiReduction</code>

Every `dot/local_dot` sums the **owned range only**, across every policy — this is the same layout-invariant rule as §3.1. `MPI_Init` must already have been called; the header defensively

defines `OMPI_SKIP_MPICXX` before including `<mpi.h>` to avoid pulling in Open MPI's deprecated C++ bindings.

`begin_sum/end` are a matched async pair for overlapping communication with computation. Only `MpiReduction` and `MpiMixedPrecisionReduction` are genuinely non-blocking (`MPI_Iallreduce`); the others implement `begin_sum` as an eager, blocking call to `sum` — callers relying on true overlap must pick one of the two MPI-async policies, not assume the interface shape implies asynchrony.

5 Solving linear systems: the Krylov API

Headers: `krylov_solvers.hpp` (GMRES/FGMRES, standard BiCGSTAB), `pipelined_bicgstab.hpp` (two communication-hiding BiCGSTAB variants), `idrs.hpp` (IDR(s)), `gcrodr.hpp` + `recycling.hpp` (GCRO-DR / augmented FGMRES).

5.1 The common contract

Every solver entry point is a free function in `owt::krylov`, templated on:

```
template<std::floating_point T, class Operator,
        class Preconditioner = IdentityPreconditioner,
        class Reduction = SerialReduction<T>>
SolverResult<T> solver_name(Operator& linear_operator, const BlockVector<T>& rhs,
                          BlockVector<T>& solution, const SolverOptions<T>& options
                          = {},
                          Preconditioner&& preconditioner = Preconditioner{},
                          Reduction reduction = {}, /* solver-specific extras */);
```

Operator must expose `void apply(BlockVector<T>&, BlockVector<T>&)`. Preconditioner must expose `void apply(const BlockVector<T>&, BlockVector<T>&)`. Both are duck-typed — no C++ concept enforces the shape.

All solves are right/flexible-preconditioned. The operator is always applied to a *pre-conditioned* direction, never the reverse; there is no left-preconditioning path anywhere in the library. This is also exactly what makes `gmres()` and `fgmres()` the same code (§5.4): a fixed preconditioner gives ordinary right-preconditioned GMRES, a preconditioner that changes state between calls gives FGMRES.

`solution` must be pre-filled by the caller with the initial guess — every solver’s first act is computing the true initial residual $\mathbf{rhs} - A\mathbf{x}_0$. `rhs/solution` must `same_layout()`; if not (or if `options` fails `valid_options()`, or `restart==0` for GMRES/FGMRES), the solver returns immediately with a default-constructed `SolverResult<T>` (`status == invalid_input`) without touching `solution`.

5.2 SolverOptions<T>

`core.hpp`.

Field	Default	Meaning
<code>relative_tolerance</code>	1e-8	$threshold = \max(absolute_tolerance, relative_tolerance \cdot \max(\ \mathbf{rhs}\ , 1))$
<code>absolute_tolerance</code>	0	Floor added to the threshold above
<code>breakdown_tolerance</code>	$64 \cdot \epsilon(T)$	Generic near-singularity threshold (Arnoldi rank loss, GMRES triangular pivots, BiCGSTAB ω collapse, IDR dense-system pivots, ...)

Field	Default	Meaning
maximum_iterations	1000	Hard cap; exceeding it returns <code>status = maximum_iterations</code>
convergence_check_interval	10	bicgstab only: how often the recursive residual is checked/replaced by the true residual
restart	30	GMRES/FGMRES restart cycle length m . 0 makes <code>fgmres</code> return immediately (no-op guard)
residual_replacement_interval	50	Iteration interval forcing a true-residual recompute + hard restart in the BiCGSTAB family. 0 disables it. Not read by <code>idrs()</code> .
idr_shadow_space	4	IDR(s) shadow-space dimension s , used only by <code>idrs()</code>
relaxation	1	Not read anywhere in the solver headers — dead field as far as <code>krylov_solvers.hpp/pipelined_bicgstab.hpp/idrs.hpp/gcr</code> are concerned (see §14; it <i>is</i> read by <code>stationary.hpp</code> 's <code>jacobi()</code>)
verify_true_residual	true	Not branched on inside the solvers reviewed here — true-residual verification happens unconditionally at every prospective-convergence point regardless of this flag
collect_timings	false	When true, allocates <code>result.timings</code> and records wall-clock solve time
gmres_orthogonalization	iterated_classic	See §5.4 gram_schmidt

`valid_options()` requires all three tolerances finite, `relative_tolerance/absolute_tolerance`
`>= 0`, `breakdown_tolerance > 0`, `maximum_iterations > 0`, `convergence_check_interval > 0`. It
does **not** validate `restart`, `idr_shadow_space`, or `relaxation`.

5.3 SolverResult<T>, SolverStatus, BreakdownReason

```
enum class SolverStatus { converged, maximum_iterations, breakdown, diverged,
    invalid_input };

struct SolverResult<T> {
    SolverStatus status = invalid_input;
    BreakdownReason breakdown_reason = none;
    std::size_t iterations = 0;
    std::size_t operator_applications = 0;
    std::size_t preconditioner_applications = 0;
    std::size_t global_reductions = 0;
    std::size_t reorthogonalizations = 0;           // FGMRES only
    T initial_residual_norm, recursive_residual_norm, true_residual_norm,
    relative_residual_norm; // NaN by default
    std::shared_ptr<SolverTimings<T>> timings;      // non-null only if collect_timings
    bool converged() const noexcept;              // status == converged
    double solve_seconds() const noexcept;        // 0 if timings == nullptr
};
```

`global_reductions` is the metric behind every “N-sync”/“N-reduction” claim in this manual —
it counts every dot/norm/sum/begin_sum-end pair. `recursive_residual_norm` is the algorithm’s

cheap formula-derived estimate; `true_residual_norm` is $\|\mathbf{rhs} - A\mathbf{x}\|$ recomputed explicitly whenever a convergence/breakdown decision needs it. Final convergence is always decided by the true residual, never the recursive estimate alone.

`BreakdownReason` (meaningful only when `status == breakdown`):

Reason	Trigger
<code>arnoldi_invariant_subspace</code>	FGMRES: Arnoldi produced a near-zero subdiagonal/Givens denominator and the cycle didn't also converge
<code>singular_projected_system</code>	FGMRES: back-substitution hit a near-zero triangular pivot
<code>biorthogonality_loss</code>	BiCGSTAB family: shadow-residual dot ρ collapsed
<code>alpha_denominator</code>	BiCGSTAB family: α 's denominator collapsed
<code>omega_denominator / omega_zero</code>	BiCGSTAB/IDR: ω 's denominator or the resulting ω collapsed
<code>idr_shadow_space</code>	<code>idrs()</code> : shadow-space construction/orthonormalization failed
<code>idr_small_system</code>	<code>idrs()</code> : the $s \times s$ moment-matrix solve was singular and shadow-space restart was already retried more than twice
<code>non_finite_scalar</code>	Any of the above checks found NaN/Inf rather than merely a tiny value

Several BiCGSTAB-family breakdown paths first check for a **happy breakdown** — the current update already drove the residual under threshold — and report `converged/diverged` instead of `breakdown` in that case.

5.4 GMRES and FGMRES

`krylov_solvers.hpp`.

```
SolverResult<T> fgmres(Operator&, const BlockVector<T>& rhs, BlockVector<T>& solution,
    const SolverOptions<T>& options = {},
    Preconditioner&& preconditioner = Preconditioner{},
    Reduction reduction = {},
    SolverWorkspace<T>* supplied_workspace = nullptr,
    ArnoldiSnapshot<T>* arnoldi_snapshot = nullptr);

SolverResult<T> gmres(/* same signature, no arnoldi_snapshot parameter */);
```

`gmres()` is a thin forwarding wrapper over `fgmres()` — algorithmically identical code; the distinction is purely whether the caller's `Preconditioner::apply` changes state between calls. `*_with_workspace` overloads take a required `SolverWorkspace<T>&` instead of the trailing pointer.

Restarted Arnoldi with a workspace sized for `restart+1` basis vectors. Orthogonalization is classical Gram-Schmidt, structured as one batched reduction combining the Hessenberg column and the pre-projection norm squared, plus a conditional second reduction controlled by `gmres_orthogonalization`:

Enumerator	Behavior
<code>iterated_classical_gram_schmidt</code>	Second reduction spent every column — two-sync CGS2, most robust, the default
<code>adaptive_classical_gram_schmidt</code>	Second reduction spent only when the Daniel–Gragg–Kaufman–Stewart norm-loss test trips ($projected_norm^2 \leq 0$ or $< 0.5 \cdot original_norm^2$)
<code>one_synchronization_classical_gram_schmidt</code>	Second reduction never spent — exact one-sync CGS, one reduction per column

The true residual is recomputed after every restart cycle; only that value decides overall convergence. If `arnoldi_snapshot` is non-null, it is populated with the unrotated Hessenberg matrix and preconditioned Krylov basis of the most recently completed cycle — this is the exact state `gcrodr()`’s harmonic-Ritz extraction consumes (§5.7).

5.5 The BiCGSTAB family

Three implementations with the same `SolverOptions/SolverResult` contract and the same `*_stable` (forces `MixedPrecisionSerialReduction<T>`), `*_residual_replacement(interval)`, `*_mixed_precision`, `*_with_workspace` wrapper naming convention — switching between them is a drop-in replacement at the call site.

Function	Header	Workspace	Reductions/Iteration	Distinguishing mechanism
<code>bicgstab</code>	<code>krylov_solvers.h</code>	9 vectors	4	Classical van der Vorst recurrence, fully synchronous, no fusing. The four reductions are ρ , α -denominator, ω -numerator, ω -denominator, as separate calls.
<code>pipelined_bicgstab</code>	<code>pipelined_bicgstab.h</code>	9 vectors	3	SpecWave legacy solver 12 port. Independent dot products fused into batched reductions; the next iteration’s $\{\rho, \text{residual norm}\}$ reduction is dispatched via <code>begin_sum</code> early, with an explicit overlap comment: “A distributed operator may update solution ghosts here without changing the recurrence.”

Function	Header	Workspace	Reductions/Dist	Distinguishing mechanism
<code>communication_hiding_bicgstab</code>	<code>pipelined_bicgstab.hpp</code>	16 vectors	2, each over- lapped with a concur- rent precondi- tioner+operator applica- tion	The PETSc/Cools 15-vector recurrence (legacy solver 15). Precomputes next-iteration search directions during each in-flight reduction, not just alongside the caller's own halo exchange.

Choosing between them: use `bicgstab` when there is no MPI latency to hide or simplicity/debuggability matters most. Use `pipelined_bicgstab` when the `Operator`'s own halo exchange can overlap with an `async` `MpiReduction` policy. Use `communication_hiding_bicgstab` when preconditioner/operator cost is comparable to or larger than reduction latency, so overlapping a reduction with a full apply is worthwhile — largest workspace, deepest hiding, most complex recurrence.

Real overlap in `pipelined_bicgstab/communication_hiding_bicgstab` only materializes with an `async` `Reduction` (`MpiReduction<T>` or `MpiMixedPrecisionReduction<T>`); under `SerialReduction<T>`, `begin_sum` is blocking and the “fusing” only reduces reduction *count*, not latency.

`bicgstab_residual_replacement(interval)` in `pipelined_bicgstab.hpp` hardcodes `SerialReduction<T>` internally, ignoring any caller reduction preference — unlike the plain-`bicgstab` residual-replacement wrapper, which forwards the caller's `Reduction` choice.

5.6 IDR(s)

`idrs.hpp`.

```
SolverResult<T> idrs(Operator&, const BlockVector<T>& rhs, BlockVector<T>& solution,
    const SolverOptions<T>& options = {},
    Preconditioner&& preconditioner = Preconditioner{},
    Reduction reduction = {});
```

`s = options.idr_shadow_space` (default 4). `s = 0` or `s > rhs.owned_size()` returns `invalid_input` immediately.

`s = 1` calls `bicgstab()` directly and returns its result verbatim — the code comment: “*IDR(1) is mathematically BiCGSTAB. Use the already guarded BiCGSTAB recurrence instead of the less robust cyclic small-system formulation.*” This is not merely a mathematical equivalence claim; it is literally a redirect to the more numerically hardened implementation.

For $s \geq 2$: shadow space is initialized by normalizing the initial residual into `shadow[0]`, then filling `shadow[1..s - 1]` with a deterministic xorshift64*-seeded pseudo-random fill, classically Gram-Schmidt-orthonormalized. An initial s -step minimum-residual (Richardson) phase establishes the first `delta_residual/delta_solution` vectors, then the main cyclic loop solves a dense $s \times s$ moment system each inner step via Gaussian elimination with partial pivoting; a singular moment system triggers up to two shadow-space restarts (labeled in-code as “*SpecWave’s numerical restart*”) before reporting `idr_small_system` breakdown.

`idrs` has **no** `SolverWorkspace` **overload** (always allocates its own vectors locally) and **no** `*_stable` `/*_mixed_precision/*_residual_replacement` **wrappers** — precision control is purely via the

Reduction template argument passed directly, and `options.residual_replacement_interval` has no effect on `idrs()` at all. It is also not communication-hiding/fused like the BiCGSTAB variants — a straightforward serial-reduction translation of SpecWave’s cyclic algorithm, costing multiple `dot/norm` calls per inner step.

5.7 Recycling and GCRO-DR

`recycling.hpp` (generic recycle infrastructure, no build flag) and `gcrodr.hpp` (harmonic-Ritz specialization, requires `OWT_KRYLOV_ENABLE_LAPACK`).

```
template<std::floating_point T>
class RecycleSpace {
    explicit RecycleSpace(std::size_t maximum_vectors); // throws if 0
    std::size_t size() const noexcept; std::size_t capacity() const noexcept;
    const BlockVector<T>& vector(std::size_t index) const; //  $U_i$ 
    const BlockVector<T>& image(std::size_t index) const; //  $C_i = A*U_i$ , kept
    globally orthonormal

    template<class Operator, class Reduction>
    bool add_candidate(Operator&, const BlockVector<T>& candidate, Reduction&,
                      SolverResult<T>* telemetry = nullptr,
                      T dependence_tolerance = 128*eps); // false if numerically
    dependent

    template<class Reduction>
    void project_initial_guess(const BlockVector<T>& residual, BlockVector<T>&
    solution,
                                Reduction&, SolverResult<T>* telemetry = nullptr); //
     $x \leftarrow U * C^{-1} * r$ 
};
```

U (search directions) and $C = AU$ (kept A -orthonormal to each other) are publicly exposed precisely so an application can inject deflation vectors from outside knowledge, not just whatever `add_candidate`’s default policy would choose. Storage is a bounded FIFO — adding past `maximum_vectors` evicts the oldest pair.

```
SolverResult<T> recycled_fgmres(Operator&, const BlockVector<T>& rhs, BlockVector<T>&
    solution,
                                RecycleSpace<T>& recycle_space, const SolverOptions<T>
    & options = {},
                                Preconditioner&& preconditioner = Preconditioner{},
                                Reduction reduction = {},
                                bool update_recycle_space = true,
                                SolverWorkspace<T>* workspace = nullptr,
                                ArnoldiSnapshot<T>* arnoldi_snapshot = nullptr);
```

Augmented FGMRES for a sequence of related systems: projects the initial guess against any existing `recycle_space` before solving, wraps the operator/preconditioner so every Arnoldi direction stays deflated against C at the cost of no *extra* operator applications on the FGMRES hot path (deflation is folded into the same apply via a one-shot memoization between the wrapped operator and preconditioner). If `update_recycle_space` (default `true`) and the solve converges, the solution correction is appended as a new recycle candidate.

```
std::size_t extract_harmonic_ritz(Operator&, const ArnoldiSnapshot<T>& snapshot,
    RecycleSpace<T>& recycle_space, Reduction&,
    SolverResult<T>* telemetry = nullptr);
```

```

SolverResult<T> gcrodr(Operator&, const BlockVector<T>& rhs, BlockVector<T>& solution,
                      RecycleSpace<T>& recycle_space, const SolverOptions<T>&
                      options = {},
                      Preconditioner&& preconditioner = Preconditioner{}, Reduction
                      reduction = {},
                      SolverWorkspace<T>* workspace = nullptr,
                      ArnoldiSnapshot<T>* supplied_snapshot = nullptr);

```

`gcrodr()` calls `recycled_fgmres(..., update_recycle_space=false, ...)`, then — if the captured snapshot has any columns — replaces `recycle_space` with harmonic-Ritz vectors of the just-completed cycle via `extract_harmonic_ritz`. The harmonic matrix is

$$H_m + h_{m+1,m}^2 H_m^{-T} e_m e_m^T,$$

solved via LAPACK `sgeev/dgeev`; complex conjugate Ritz pairs contribute both their real and imaginary invariant-subspace directions; the smallest-magnitude eigenvalues are retained first (the textbook choice for deflating the slowest-converging directions).

Use `recycled_fgmres` directly when the default “recycle the converged correction” policy is enough (e.g. repeated similar right-hand sides against a fixed operator). Use `gcrodr` when you want spectrally-informed deflation across genuinely related but not identical operators (e.g. successive Newton/time-step Jacobians).

5.8 Reusable workspaces

`SolverWorkspace<T>` (`core.hpp`) holds a pool of `BlockVector<T>` and a pool of scalars. `prepare(layout, vector_count, scalar_count)` reallocates the **entire** vector pool if the layout changed or the pool is smaller than requested — even growing `vector_count` with an unchanged layout throws away and rebuilds the whole pool, invalidating any previously obtained references. `allocation_epochs()` counts reallocation events, letting a caller confirm a workspace is being reused with zero allocation across repeated solves — the main reason to pass a `SolverWorkspace*` explicitly rather than let each call build (and destroy) its own.

`ArnoldiSnapshot<T>` (`core.hpp`) captures the unrotated Hessenberg matrix and preconditioned Krylov basis of the most recently completed FGMRES cycle — see §5.4 and §5.7.

6 Preconditioners

`preconditioner.hpp` (rank-local baselines + genuine RAS), `split_block_csr.hpp` (`SplitLocalSsorPreconditioner`), `stationary.hpp` (full iterative solvers usable as smoothers), `overlap.hpp/overlap_plan.hpp` (MPI overlap discovery feeding RAS).

Two conventions recur across every preconditioner type:

- `apply(const BlockVector<T>&, BlockVector<T>&) const` is the universal contract — duck-typed, no shared base class. Anything with this signature plugs directly into any solver in §5.
- **Symbolic vs. numeric setup**: sparsity/size (“symbolic”) state is fixed at construction; a coefficient-only refresh (“numeric”) is a separate, explicitly-called method that re-validates the pattern is unchanged and throws if it isn’t — never silently rebuilds.

6.1 Identity and Jacobi

```
struct IdentityPreconditioner { void apply(const BlockVector<T>&, BlockVector<T>&)
    const; }; // copy_owned, pass-through

template<std::floating_point T>
class JacobiPreconditioner {
    template<class Matrix> explicit JacobiPreconditioner(const Matrix&, T
        minimum_diagonal = 64*eps);
    void update_values(const Matrix&); // re-validates layout, recomputes inverse
        diagonal
    void apply(const BlockVector<T>&, BlockVector<T>&) const;
    std::size_t numeric_updates() const noexcept;
};
```

`Matrix` must expose `owned_nodes()/ghost_nodes()/block_size()` and `diagonal()` -> `std::vector<T>` — works with `BlockCsrMatrix` or `DenseBlockCsrMatrix`. Throws `std::invalid_argument` if any diagonal entry is non-finite or $|value| \leq \text{minimum_diagonal}$.

6.2 Local SSOR

Two independent implementations, one per matrix storage form.

```
template<std::floating_point T, std::integral Index = std::size_t>
class LocalSsorPreconditioner { // block_csr.hpp companion, in preconditioner.hpp
    explicit LocalSsorPreconditioner(const BlockCsrMatrix<T, Index>& matrix, T omega =
        T(1)); // 0<omega<2
    void update_values(); // re-reads the diagonal through the stored matrix
        pointer
    void apply(const BlockVector<T>&, BlockVector<T>&) const;
};
using RasSsorPreconditioner = LocalSsorPreconditioner<...>; // compatibility alias,
    see Sec 14
```

Doc comment: “Zero-overlap rank-local SSOR. Off-rank/ghost entries are deliberately excluded from the triangular solves. This is the overlap-zero baseline for Schwarz methods; it is not an overlapping RAS preconditioner by itself.” Both forward and backward sweeps unconditionally skip any `column ≥ owned_nodes()` — that’s the entire mechanism of “zero overlap.” `LocalSsorPreconditioner` stores a raw pointer to the matrix (must outlive it); `update_values()` takes **no argument** (re-reads through the stored pointer) — an asymmetry versus `JacobiPreconditioner` / `Ilu0Preconditioner`, whose `update_values(matrix)` takes an explicit argument.

```
template<std::floating_point T, std::integral Index = std::size_t>
class SplitLocalSsorPreconditioner { // split_block_csr.hpp
    explicit SplitLocalSsorPreconditioner(const SplitBlockCsrMatrixView<T, Index>&, T
        omega = T(1));
    void update_values(const SplitBlockCsrMatrixView<T, Index>&);
    void apply(const BlockVector<T>&, BlockVector<T>&) const;
};
```

Same zero-overlap SSOR sweep, adapted to the split diagonal/off-diagonal layout. Zero-copy: never copies the application’s diagonal/off_diagonal arrays. Writes only output’s owned region — the ghost region is left whatever it was before the call.

6.3 ILU(0)

```
template<std::floating_point T, std::integral Index = std::size_t>
class Ilu0Preconditioner {
    explicit Ilu0Preconditioner(const BlockCsrMatrix<T, Index>& matrix);
    void update_values(const BlockCsrMatrix<T, Index>& matrix);
    void apply(const BlockVector<T>&, BlockVector<T>&) const;
    std::size_t numeric_updates() const noexcept;
};
```

“Rank-local block-Jacobi ILU(0), vectorized over every node component.” Construction drops all ghost couplings (`column < owned_nodes()` only) and records, per retained entry, where it came from in the caller’s matrix — this is the symbolic pattern. `update_values(matrix)` re-checks that every row’s local-entry count and every retained entry’s source column are unchanged, throwing “ILU(0) sparsity pattern changed” otherwise; a structurally different matrix requires a new `Ilu0Preconditioner`, not a refresh. Factorization is genuine no-fill ILU(0) — any elimination fill target not already in the pattern is silently dropped, never inserted. Throws `std::runtime_error("zero pivot in ILU(0)")` on a pivot with magnitude $\leq 64 \cdot \text{eps}$.

This is the internal local solver used by both RAS preconditioners below.

6.4 Restricted Additive Schwarz

Two variants: a one-shot depth-1 builder plus preconditioner (`overlap.hpp` + `preconditioner.hpp`), and an arbitrary-depth, cached version (`overlap_plan.hpp`) for repeated numeric refresh on a fixed sparsity.

Depth-1, uncached

```
struct DepthOneOverlap<T> { BlockCsrMatrix<T, std::size_t> matrix; std::vector<std::size_t> outer_local_nodes; };

DepthOneOverlap<T> build_depth_one_overlap(MPI_Comm, const DistributedLayout&
```

```

                                const BlockCsrMatrix<T,Index>&
local_matrix);

template<std::floating_point T, std::integral Index, class Halo>
class RestrictedAdditiveSchwarzIlu0 {
    RestrictedAdditiveSchwarzIlu0(const BlockVector<T>& outer_layout,
                                BlockCsrMatrix<T,Index> overlap_matrix,
                                std::vector<std::size_t> outer_local_nodes,
                                Halo& residual_halo, std::size_t overlap_depth);
    void apply(const BlockVector<T>&, BlockVector<T>&) const;
    BlockCsrMatrix<T,Index>& overlap_matrix(); // mutate coefficients directly, then
    :
    void update_values(); // re-factorize the internal
    Ilu0Preconditioner
    std::size_t overlap_depth() const noexcept; // pure metadata, never used
    algorithmically
};

```

`build_depth_one_overlap` is a full MPI collective: for every ghost node, it requests that node's complete owned CSR row from its owner (`MPI_Alltoall/MPI_Alltoallv`), then discards any received entry whose column falls outside this rank's own owned+ghost set — the literal mechanism of “restricted.” Every call redoes the full exchange; there is no numeric-only fast path.

`RestrictedAdditiveSchwarzIlu0::apply`: refreshes ghosts on a synchronized copy of the residual via `residual_halo.exchange`, gathers the local RHS via the `outer_local_nodes` map (restriction), solves locally with an internal `Ilu0Preconditioner`, then scatters the correction back **only for subdomain rows whose outer node is in the owned range** — discarding corrections computed in the overlap region is the “restricted” in RAS. Updating coefficients is two manual steps: `mutate_overlap_matrix()` directly, then call `update_values()`.

Arbitrary depth, cached

```

template<std::floating_point T, std::integral Index = std::size_t>
class DistributedOverlapPlan {
    DistributedOverlapPlan(MPI_Comm, const DistributedLayout&,
                          const BlockCsrMatrix<T,Index>& local_matrix, std::size_t
depth); // depth >= 1
    std::size_t depth() const; std::size_t subdomain_nodes() const;
    const BlockCsrMatrix<T,std::size_t>& matrix() const;
    void update_values(const BlockCsrMatrix<T,Index>& local_matrix); // numeric-only
    , cached
    void gather_residual(const BlockVector<T>& outer, BlockVector<T>& subdomain_rhs)
    const;
};

template<std::floating_point T, std::integral Index = std::size_t>
class CachedRestrictedAdditiveSchwarzIlu0 {
    CachedRestrictedAdditiveSchwarzIlu0(const BlockVector<T>& outer_layout,
    DistributedOverlapPlan<T,Index>& plan);
    void apply(const BlockVector<T>&, BlockVector<T>&) const;
    void update_values(const BlockCsrMatrix<T,Index>& local_matrix); // plan.
    update_values() + re-factorize
    std::size_t overlap_depth() const;
};

```

DistributedOverlapPlan's constructor does the expensive one-time work: a frontier-based BFS over depth graph layers (via MPI_Alltoall/MPI_Alltoallv), discovering exactly which ranks to talk to, how many entries, and which local slots to read/write — then caches all of it. update_values() afterward costs only local copies plus exactly depth Alltoallv calls per numeric refresh — no id or structure exchange, no graph traversal. gather_residual() uses a **separately pre-cached** one-round-trip schedule (Alltoallv regardless of depth) — fetching a residual value only needs its owner directly, unlike coefficient exchange, which must walk outward layer by layer.

Overlap depth = number of BFS layers imported beyond the outer ghost layer already present in DistributedLayout. **depth** = 1 is comparable in reach to build_depth_one_overlap (not necessarily byte-identical); **depth** = 2 additionally imports rows for nodes newly discovered via depth-1 rows' off-rank columns; and so on. Larger depth generally improves RAS convergence at the cost of more per-update communication and a larger local ILU(0) subdomain. Pick the smallest depth that gives acceptable Krylov iteration counts.

Prefer DistributedOverlapPlan + CachedRestrictedAdditiveSchwarzIlu0 over build_depth_one_overlap + RestrictedAdditiveSchwarzIlu0 whenever depth > 1 is needed, or the same sparsity will be refactorized repeatedly (nonlinear iterations, time steps). A structural sparsity change still requires a new plan (and a new CachedRestrictedAdditiveSchwarzIlu0) either way.

6.5 Stationary methods and smoothers

stationary.hpp. These are complete solvers (SolverResult<T>, own convergence loop), not bare apply(input,output) preconditioners — they can be called standalone, or (in SpecWave's usage) as full solves for legacy solver IDs 0/1/2/16.

```
std::vector<T> chebyshev_srj_schedule(std::size_t levels, T spectral_radius = 0.99);

SolverResult<T> relaxed_jacobi(Operator&, rhs, solution, options,
    diagonal_preconditioner,
                                std::span<const T> relaxation_schedule, reduction =
    {});
SolverResult<T> jacobi(Operator&, rhs, solution, options, diagonal_preconditioner,
    reduction = {});
SolverResult<T> chebyshev_srj(Operator&, rhs, solution, options,
    diagonal_preconditioner,
                                std::size_t levels = 8, T spectral_radius = 0.99,
    reduction = {});

SolverResult<T> gauss_seidel(BlockCsrMatrix<T,Index>&, Operator&, Halo&, rhs, solution
    , options, reduction = {});
SolverResult<T> asynchronous_gauss_seidel(/* same signature */);
SolverResult<T> anderson_jacobi(Operator&, rhs, solution, options,
    diagonal_preconditioner,
                                std::size_t period = 3, T mixing = T(1), reduction =
    {});
```

- jacobi/chebyshev_srj both wrap relaxed_jacobi with a relaxation schedule (a single-element {options.relaxation} for plain Jacobi — this is where SolverOptions::relaxation is actually read, unlike in the Krylov solvers of §5; a Chebyshev-SRJ schedule for the other). The schedule applies **cyclically**: maximum_iterations should ideally be a multiple of levels to complete whole Chebyshev cycles.

- `gauss_seidel` is synchronous: `halo.exchange(solution)` blocks every iteration before a full forward sweep over all owned rows.
- `asynchronous_gauss_seidel` overlaps: `halo.begin(solution)` (non-blocking), update `matrix.interior_rows()` while it's in flight, `halo.end(handle)`, then update `matrix.boundary_rows()`. This is the “AsyncGS” of the README/port matrix (legacy solver 16).
- `anderson_jacobi` is SpecWave's type-I, depth-one Anderson acceleration of Jacobi: every `period`-th iteration (once one prior fixed-point/defect pair exists), mixes the current and previous fixed-point iterates with a coefficient $\gamma = \langle f_k, \Delta f \rangle / \langle \Delta f, \Delta f \rangle$ clamped to $[-2, 2]$, only when $\langle \Delta f, \Delta f \rangle > \text{breakdown_tolerance}$. `period` controls how often mixing is attempted, not how much history is kept — depth is fixed at 1.

None of these functions are called by `AggregationMultigrid` (§7.3), which reimplements its own inline Jacobi/red-black-GS/Chebyshev smoother sweeps privately rather than reusing this header.

7 Coarse levels and multigrid

`coarse.hpp` (MPI-collective, gated on `OWT_KRYLOV_ENABLE_MPI`), `multigrid.hpp` (no MPI dependency), `petsc_coarse.hpp` (gated on `OWT_KRYLOV_ENABLE_PETSC`, requires MPI).

7.1 SubdomainConstantCoarseCorrection

```
template<std::floating_point T, std::integral Index = std::size_t>
class SubdomainConstantCoarseCorrection {
    SubdomainConstantCoarseCorrection(MPI_Comm, const DistributedLayout&, const
    BlockCsrMatrix<T, Index>&);
    void update_values(const BlockCsrMatrix<T, Index>&); // full re-derivation +
    MPI_Allreduce, every call
    void apply(const BlockVector<T>&, BlockVector<T>&) const;
    std::size_t coarse_dimension() const; // rank_count * block_size
};
```

Doc comment: “*Piecewise-constant coarse correction with one aggregate per MPI rank and component. The Galerkin operator $A_c = P^T A P$ is assembled collectively... and solved by a replicated, pivoted dense LU factorization. This is intentionally a robust baseline coarse level for modest rank counts.*” The coarse space has exactly one DOF per component per rank; every node owned by a rank shares one constant coarse value per component. Construction and every `update_values/apply` call is a full MPI collective — `Allreduce` cost, $O(\text{rank_count}^2)$ replicated dense storage/factorization per component. There is **no** structural short-circuit like the local preconditioners’ symbolic/numeric split — `update_values()` re-walks the entire local sparsity and re-does the `Allreduce` every time.

The README is explicit: this remains useful only as an inspectable small-rank baseline; production scale should select `PetscSubdomainCoarseCorrection` (§7.4).

7.2 MultiplicativeTwoLevelPreconditioner

```
template<class T, class Local, class Coarse, class Operator>
class MultiplicativeTwoLevelPreconditioner {
    MultiplicativeTwoLevelPreconditioner(Local&, Coarse&, Operator&, const BlockVector
    <T>& layout);
    void apply(const BlockVector<T>&, BlockVector<T>&) const;
};
```

Composes any coarse-level type (§7.1 or §7.4) with any local preconditioner (§6) and the fine-level operator:

$$M^{-1}r = \text{coarse}(r) + \text{local}(r - A \cdot \text{coarse}(r))$$

— genuinely multiplicative, one extra operator application per call (tracked separately via `internal_operator_applications()`, not auto-folded into a `SolverResult`). Holds only non-owning pointers to `local/coarse/matrix_operator`; the caller must `update_values()` each of them independently — the composer has no `update` method of its own. Satisfies the same `apply(input,output)` contract as everything else in §6, so it plugs directly into any solver in §5.

7.3 AggregationMultigrid

`multigrid.hpp`. A self-contained two-level (fine + one coarse) aggregation multigrid **solver**, not a bare preconditioner — the coarse solve is explicitly rank-local; the header’s own comment: “a distributed coarse-space implementation is a separate required scalability milestone.”

```
enum class MultigridCycle { v, w, full };
enum class MultigridSmoother { jacobi, red_black_gauss_seidel, chebyshev };
struct MultigridOptions {
    std::size_t pre_smoothing_steps = 2, post_smoothing_steps = 2,
    coarse_smoothing_steps = 24;
    std::size_t target_aggregate_size = 4; double jacobi_weight = 0.8;
    MultigridSmoother smoother = jacobi; std::size_t chebyshev_levels = 8;
};

template<std::floating_point T, std::integral Index = std::size_t>
class AggregationMultigrid {
    AggregationMultigrid(const BlockCsrMatrix<T, Index>& fine_matrix, MultigridOptions
    = {});
    SolverResult<T> solve(Operator& fine_operator, const BlockVector<T>& rhs,
    BlockVector<T>& solution,
                        const SolverOptions<T>&, MultigridCycle cycle = v,
    Reduction reduction = {}) const;
};
```

Everything — greedy graph aggregation, Galerkin coarse-operator assembly, diagonal caching, a heuristic 2-coloring for red-black smoothing — is done once in the constructor; **there is no numeric-only refresh method at all**. A coefficient change on the fine matrix requires constructing a brand-new `AggregationMultigrid`. Aggregation only considers `column < owned_nodes()` couplings (never crosses rank boundaries) and is greedy in CSR-entry order, not strength-of-connection based.

`MultigridCycle::w` here means literally “do the coarse-grid correction twice per fine iteration” (there is no third level to recurse into) rather than a true recursive W-cycle. `MultigridCycle::full` only changes the *initial guess* (one nested-iteration FMG pass before the main loop begins) — the subsequent loop is otherwise identical to `v`. `coarse_smoothing_steps` defaults much higher (24) than `pre/post_smoothing_steps` (2) because the coarse level here is only iterative relaxation, not a direct solve (contrast §7.1’s dense LU factorization) — raise it, or shrink `target_aggregate_size`, if the coarse level under-damps the smooth error mode.

The `Operator` passed to `solve()` is used only for fine-level applies (so it can carry a halo exchange); the coarse level is always solved through the class’s own internally-stored, rank-local coarse matrix with no `Operator`/halo abstraction at that level.

7.4 PetscSubdomainCoarseCorrection

See §9.2.

8 Distributed execution (MPI)

Everything in this chapter requires `-DOWT_KRYLOV_ENABLE_MPI=ON`. OWT never calls `MPI_Init`/`MPI_Finalize` — the application must do this before constructing any of these types.

8.1 `MpiHaloExchange<T>`

`mpi_halo.hpp`. A zero-copy, reusable halo-exchange plan built from **committed derived MPI datatypes**.

```
struct Neighbor { int rank = -1; std::vector<std::size_t> send_owned_nodes,
    receive_ghost_nodes; };

template<std::floating_point T>
class MpiHaloExchange {
    MpiHaloExchange(MPI_Comm, std::size_t owned_nodes, std::size_t ghost_nodes,
        std::size_t block_size, std::vector<Neighbor>, int tag = 9173);
    struct Handle { std::vector<MPI_Request> requests; };
    Handle begin(BlockVector<T>&) const;    // posts non-blocking Irecv then Isend per
    neighbor
    void end(Handle&) const;                // MPI_Waitall
    void exchange(BlockVector<T>&) const;   // begin + end
};
```

For each `Neighbor`, one `MPI_Type_create_indexed_block` is built and committed per non-empty direction (send/receive) at construction — a single `MPI_Irecv`/`MPI_Isend` per neighbor per `begin()` moves the whole scattered node set directly out of `vector.data()`, no staging buffer. Every neighbor must have `rank ≥ 0` and at least one non-empty node list, or construction throws. `block_size` and every computed displacement (`node_index · block_size`) must fit in `int` — the MPI count type — so total per-rank vector size is capped at `INT_MAX` elements. Element type is `MPI_FLOAT` for `T=float`, `MPI_DOUBLE` for `T=double`; other floating types are not actually exchangeable despite the class template only constraining `std::floating_point`. The destructor checks `MPI_Finalized` before freeing committed datatypes, to avoid calling into a torn-down MPI runtime.

8.2 `OverlappedDistributedBlockOperator`

`mpi_halo.hpp`.

```
template<class Matrix, class Halo>
class OverlappedDistributedBlockOperator {
    OverlappedDistributedBlockOperator(Matrix matrix, Halo halo);
    template<std::floating_point T> void apply(BlockVector<T>& input, BlockVector<T>&
        output);
};
```

`apply` overlaps communication with computation explicitly:

1. `halo_.begin(input)` — post non-blocking send/recv.

2. `matrix_.apply_rows(input, output, matrix_.interior_rows())` — compute rows independent of ghost data while the exchange is in flight.
3. `halo_.end(handle)` — block until ghost data has arrived.
4. `matrix_.apply_rows(input, output, matrix_.boundary_rows())` — compute the rest.

A deduction guide binds the matrix by reference when constructed from an lvalue (zero-copy for the matrix; the halo policy is stored by value, moved in).

8.3 Overlap discovery for RAS

Covered in §6.4: `build_depth_one_overlap` (`overlap.hpp`) and `DistributedOverlapPlan` (`overlap_plan.hpp`). Both are MPI collectives; every rank in the communicator must call them, even a rank with an empty local frontier, since it may still need to serve another rank's request.

8.4 Choosing a reduction policy

See §4.4. For a genuinely distributed solve, pass `MpiReduction<T>(communicator)` (or `MpiMixedPrecisionReduction<T>` for `T=float` solves needing extended-precision global sums) as the `Reduction` template argument to any solver in §5. `MpiDeterministicReduction<T>` exists for correctness verification (bit-reproducible for a fixed rank count and partition), not for production performance measurement.

9 PETSc adapter

`petsc_adapter.hpp` and `petsc_coarse.hpp`, both gated on `OWT_KRYLOV_ENABLE_PETSC` (which itself requires `OWT_KRYLOV_ENABLE_MPI`). Every PETSc call is wrapped in `detail::petsc_check(...)`, which throws `std::runtime_error` on any non-PETSC_SUCCESS return code.

9.1 PetscBlockCsrSolver

```
struct PetscSolverOptions {
    std::string ksp_type = KSPGMRES, pc_type = PCBJACOBI, options_prefix = "owt_";
    PetscReal relative_tolerance = 1e-8, absolute_tolerance = PETSC_DEFAULT;
    PetscInt maximum_iterations = 1000; bool nonzero_initial_guess = true;
    KSPNormType norm_type = KSP_NORM_UNPRECONDITIONED;
};

template<std::floating_point T, std::integral Index = std::size_t>
class PetscBlockCsrSolver {
    PetscBlockCsrSolver(MPI_Comm, const DistributedLayout&, const BlockCsrMatrix<T,
    Index>&,
                        PetscSolverOptions = {});
    void update_values(); // structure-preserving numeric refresh
    SolverResult<T> solve(const BlockVector<T>& rhs, BlockVector<T>& solution);
    double structure_setup_seconds() const; double last_numeric_update_seconds() const
    ;
    std::size_t numeric_updates() const;
};
```

Build-time constraints: `static_assert(sizeof(T) == sizeof(PetscScalar))` — mixing e.g. `double` OWT with a single-precision PETSc build fails to compile — and complex `PetscScalar` builds are rejected. The adapter stores raw (borrowed) pointers to the `DistributedLayout` and `BlockCsrMatrix` passed at construction; both must outlive the adapter.

Hard precondition on `DistributedLayout`: at construction, every local owned node’s `algebraic_local_to_global()` entry must equal exactly the contiguous global row PETSc’s own `MatCreateAIJ` ownership range assigns that rank (`std::invalid_argument("PETSc adapter requires rank-contiguous algebraic owned numbering")` otherwise) — i.e. the layout’s global numbering must already be partitioned in rank order matching PETSc’s row distribution.

`solve()`: copies `rhs/solution` into PETSc `Vecs` (a real copy, not zero-copy, unlike the halo exchange), calls `KSPSolve`, then **independently recomputes the true residual** via `MatMult + VecAYPX` against the actually assembled matrix rather than trusting `KSPGetResidualNorm` alone — this is the “independent true-residual checking” referenced throughout the README and docs/. `timings` is always allocated (unconditionally, unlike the native solvers, which only allocate when `SolverOptions::collect_timings` is set).

`update_values()` is a structure-preserving numeric-only refresh (`MatZeroEntries + re-insert` via cached preallocation) — call it after mutating the source matrix’s coefficients (e.g. each nonlinear/time-step iterate) without rebuilding the PETSc `Mat`.

9.2 PetscSubdomainCoarseCorrection

petsc_coarse.hpp.

```
struct PetscCoarseOptions {
    std::string ksp_type = KSPGMRES, pc_type = PCGAMG, options_prefix = "owt_coarse_";
    PetscReal relative_tolerance = 1e-10; PetscInt maximum_iterations = 200;
};

template<std::floating_point T, std::integral Index = std::size_t>
class PetscSubdomainCoarseCorrection {
    PetscSubdomainCoarseCorrection(MPI_Comm, const DistributedLayout&, const
    BlockCsrMatrix<T, Index>&,
                                PetscCoarseOptions = {});
    void update_values(const BlockCsrMatrix<T, Index>&);
    void apply(const BlockVector<T>&, BlockVector<T>&) const; // throws on
    divergence, unlike solve()
    std::size_t coarse_dimension() const; // rank_count * block_size -- still global
    O(P)
    std::size_t local_nonzeros() const; // owners_.size() * block_size -- O(
    neighbor-rank degree), not O(P^2)
};
```

Doc comment: “Sparse distributed version of the rank-constant Galerkin coarse correction. Each rank owns exactly `block_size` coarse rows and stores couplings only to rank aggregates touched by its local unstructured matrix. It therefore removes the $O(P^2 \cdot \text{block_size})$ replicated storage/factorization of the inspectable baseline and delegates the coarse solve to PETSc.” This is the scalable replacement the README recommends over `SubdomainConstantCoarseCorrection` for production use: the *global* coarse-space size is still $O(\text{rank_count})$, but **local** storage/nonzero count only scales with how many distinct neighbor ranks actually touch this rank’s local matrix — a one-time `analyze_structure()` pass at construction discovers exactly that neighbor set, and every restriction/prolongation is still piecewise-constant (sum-owned-nodes restriction, broadcast prolongation), matching the coarse-space semantics of §7.1.

`apply()` throws `std::runtime_error("distributed sparse coarse solve diverged")` on a non-converged `KSPConvergedReason` — stricter than `PetscBlockCsrSolver::solve()`, which returns a diverged `SolverResult` instead. The default `pc_type` here is `PCGAMG` (algebraic multigrid), versus `PCBJACOBI` for the fine-level adapter — the coarse solve is meant to be solved more thoroughly by default.

10 SpecWave compatibility layer

specwave_compat.hpp. No `#ifdef` gate on the file itself, though it `#includes` `idrs.hpp`, `multigrid.hpp`, `pipelined_bicgstab.hpp`, `stationary.hpp` (not `krylov_solvers.hpp` — a translation unit calling `gmres/bicgstab` directly alongside this header must include `krylov_solvers.hpp` itself).

10.1 SpecWaveSolver enum and dispatcher

```
enum class SpecWaveSolver : int { jacobi = 0, gauss_seidel, chebyshev_srj,
    pipelined_stable, pipelined_residual_replacement, bicgstab_ssor, petsc,
    gmres_ssor, bicgstab_pure_ssor, bicgstab_ilu0, pipelined_mixed_precision,
    bicgstab_full_system, pipelined_full_system, bicgstab_full_system_ilu0,
    idrs, communication_hiding_bicgstab, asynchronous_gauss_seidel,
    multigrid_v_jacobi, multigrid_v_red_black, multigrid_w, full_multigrid,
    pipelined_bicgstab_ilu0 };    // = 21

constexpr std::string_view name(SpecWaveSolver) noexcept;

template<std::floating_point T, std::integral Index, class Operator, class Halo,
    class Reduction = SerialReduction<T>>
SolverResult<T> solve_specwave_native(SpecWaveSolver, const BlockCsrMatrix<T,Index>&
    matrix,
                                Operator&, Halo&, const BlockVector<T>& rhs,
                                BlockVector<T>& solution, SolverOptions<T>
    options = {},
                                Reduction reduction = {});
```

Doc comment: “Compatibility dispatcher for every native SpecWave solver ID. PETSc remains an explicit optional adapter because it requires runtime and numbering state.”

ID	Enumerator	Dispatches to	Preconditioner
0	<code>jacobi</code>	<code>jacobi(...)</code>	<code>JacobiPreconditioner</code>
1	<code>gauss_seidel</code>	<code>gauss_seidel(matrix, op, halo, ...)</code>	--
2	<code>chebyshev_srj</code>	<code>chebyshev_srj(..., 8, 0.99, ...)</code>	<code>JacobiPreconditioner</code> ; 8 stages, 0.99 hard-coded
3	<code>pipelined_stable</code>	<code>communication_hiding_bicgstab(...)</code>	<code>LocalSsorPreconditioner</code>
4	<code>pipelined_residual_replacement</code>	same as with residual replacement interval forced to 50 if it was 0	<code>LocalSsorPreconditioner</code>
5, 8, 11	<code>bicgstab_ssor</code> , <code>bicgstab_pure_ssor</code> , <code>bicgstab_full_system</code>	<code>bicgstab(...)</code> — all three share one case label, dispatched identically	<code>LocalSsorPreconditioner</code>
6	<code>petsc</code>	returns {} unconditionally — deliberate no-op	--

ID	Enumerator	Dispatches to	Preconditioner
7	gmres_ssor	gmres(...)	LocalSsorPreconditioner
9, 13	bicgstab_ilu0, bicgstab_full_system_ilu0	bicgstab(...), shared case	Ilu0Preconditioner
10	pipelined_mixed_precision	communication_hiding_bicgstab(...) with a compile-time-selected mixed-precision reduction (see below)	LocalSsorPreconditioner
12	pipelined_full_system	pipelined_bicgstab(...)	LocalSsorPreconditioner
14	idrs	idrs(...)	LocalSsorPreconditioner
15	communication_hiding_bicgstab	communication_hiding_bicgstab(...)	LocalSsorPreconditioner
16	asynchronous_gauss_seidel	asynchronous_gauss_seidel(matrix, op, halo, ...)	--
17-20	multigrid_v_jacobi, multigrid_v_red_black, multigrid_w, full_multigrid	builds an AggregationMultigrid and calls .solve(...); smoother = red-black only for ID 18, else Jacobi; cycle = w/full for IDs 19/20, else v	--
21	pipelined_bicgstab_ilu0	pipelined_bicgstab(...)	Ilu0Preconditioner
any un- matched	--	returns {}	--

Only `gauss_seidel` (ID 1) and `asynchronous_gauss_seidel` (ID 16) actually use the `halo` parameter — every other case ignores it.

ID 10's reduction selection (`pipelined_mixed_precision`) is the one branch with an internal `#ifdef OWT_KRYLOV_ENABLE_MPI`: if the caller's `Reduction` is `MpiReduction<T>`, it's upgraded to `MpiMixedPrecisionReduction<T>(reduction.communicator())`; if it's already `MpiMixedPrecisionReduction<T>`, it passes through; otherwise (serial) it uses `MixedPrecisionSerialReduction<T>{}`.

SpecWaveSolver::petsc (ID 6) is a deliberate no-op — callers needing ID 6 must special-case it and call `PetscBlockCsrSolver` directly (§9.1).

10.2 ID-to-implementation table

From `docs/SPECWAVE_PORT_MATRIX.md`.

Legacy ID	SpecWave implementation	OWT-Krylov implementation	Status
0	Jacobi loop, <code>RDschemes_implicit_V2.hpp</code>	<code>jacobi</code>	Generic operator and diagonal preconditioner
1	Gauss-Seidel loop	<code>gauss_seidel</code>	Fused node blocks and halo policy
2	<code>ChebyshevSRJ.hpp</code>	<code>chebyshev_srj</code>	Generated Chebyshev schedule

Legacy ID	SpecWave implementation	OWT-Krylov implementation	Status
3	<code>solve_pipelined_stable</code>	<code>communication_hiding_bicgstab</code> + compensated/mixed reduction	Deterministic global reduction order not guaranteed by MPI
4	<code>solve_pipelined_rr</code>	<code>communication-hiding</code> recurrence + residual-replacement	Verified true residual on replacement/convergence
5	<code>BiCGSTAB_Solver::solve</code> , SSOR	<code>bicgstab</code> + <code>LocalSsorPreconditioner</code>	Zero-overlap local solve
6	<code>PETScFullSystemSolver</code>	<code>PetscBlockCsrSolver</code>	Optional adapter; no global-ID all-gather; independent true residual
7	<code>GMRES_Solver::solve</code> , SSOR	<code>gmres/fgmres</code> + zero-overlap local SSOR	FGMRES is an added generalization
8	Same core path as ID 5	same as ID 5	Preserved compatibility alias
9	<code>solve_ilu0</code>	<code>bicgstab</code> + <code>Ilu0Preconditioner</code>	Rank-local block-Jacobi ILU(0)
10	15-vector <code>solve_pipelined_mixed</code>	<code>communication_hiding_bicgstab</code> + mixed-precision reduction	Ported separately from ID 12; MPI float scalars reduce through double
11	<code>solve_full_system</code>	distributed <code>bicgstab</code>	Ownership-aware global reductions are policies
12	full-system <code>solve_pipelined</code>	<code>pipelined_bicgstab</code>	Normal three-reduction recurrence with happy-breakdown check
13	full-system <code>solve_ilu0</code>	distributed <code>bicgstab</code> + ILU(0)	—
14	<code>IDRs_Solver</code>	<code>idrs</code>	Cyclic dR/dX form; IDR(1) uses equivalent BiCGSTAB recurrence
15	PETSc/Cools-style 15-vector <code>solve_pipelined</code>	<code>communication_hiding_bicgstab</code>	Ported separately from ID 12; two batched reductions overlap preconditioner/operator work
16	<code>AsyncGS_Solver</code>	<code>asynchronous_gauss_seidel</code>	Interior/boundary ordering with begin/end halo exchange
17	aggregation V-cycle, Jacobi	AggregationMultigrid, V, Jacobi	Coarse solve remains rank-local
18	aggregation V-cycle, red-black GS	AggregationMultigrid, V, red-black	Coarse solve remains rank-local
19	aggregation W-cycle	AggregationMultigrid, W	Two-level cycle
20	full multigrid	AggregationMultigrid, full	Two-level FMG initialization and polishing
21	<code>solve_ilu0_pipelined</code>	<code>pipelined_bicgstab</code> + ILU(0)	Normal three-reduction recurrence

Legacy ID	SpecWave implementation	OWT-Krylov implementation	Status
22	OWT integration path	OWTKrylovSpecWaveSolver + borrowed split CSR	See caveat below

SpecWave’s optional `AAJ_Solver` is ported as `anderson_jacobi` (`stationary.hpp`, §6.5); it is not a separate numeric legacy ID.

ID 22 caveat: `OWTKrylovSpecWaveSolver` and `solver_type=22` are **not defined anywhere in this repository** — confirmed by exhaustive search of `include/`, `tests/`, `benchmarks/`. Per the port matrix, ID 22 is “the OWT integration path” and lives on the **SpecWave side** (the sibling application repository), which consumes this library’s `SplitBlockCsrMatrixView`, `MpiHaloExchange`, etc. by reference, but that class is not part of OWT-Krylov’s public API. Do not treat it as callable from this library.

ID 11 discrepancy: the port matrix table describes ID 11 as mapping to “distributed `bicgstab`” without mentioning SSOR, but `solve_specwave_native`’s actual code shares ID 11’s case label with IDs 5 and 8 — all three get the identical `bicgstab + LocalSsorPreconditioner` call. This manual documents the code’s actual behavior, not the table’s description, where they disagree.

11 Complete examples

All five are adapted from the actual test suite (cited per example) and compile against the API documented above.

11.1 Serial GMRES

Based on tests/test_core.cpp.

```
#include <owt/krylov/owt_krylov.hpp>
#include <iostream>

int main() {
    using namespace owt::krylov;

    BlockCsrMatrix<double> matrix(
        /*owned=*/3, /*ghost=*/0, /*block_size=*/2,
        /*row_offsets=*/{0, 2, 5, 7},
        /*columns=*/{0, 1, 0, 1, 2, 1, 2},
        /*values=*/{4, 5, -1, -1, -1, -1, 4, 5, -1, -1, -1, -1, 3, 4});
    DistributedBlockOperator operator_view(matrix); // zero-copy: borrows matrix

    BlockVector<double> exact(3, 0, 2);
    for (std::size_t i = 0; i < exact.owned_size(); ++i) exact.data()[i] = double(i + 1);
    BlockVector<double> rhs = exact.clone_layout();
    BlockVector<double> exact_copy = exact;
    operator_view.apply(exact_copy, rhs); // manufacture a consistent rhs

    SolverOptions<double> options;
    options.relative_tolerance = 1e-12;
    options.convergence_check_interval = 1;
    options.maximum_iterations = 100;
    options.restart = 8;
    const JacobiPreconditioner jacobi(matrix);

    BlockVector<double> solution = exact.clone_layout();
    const auto result = gmres(operator_view, rhs, solution, options, jacobi);
    std::cout << "converged=" << result.converged()
               << " iterations=" << result.iterations << '\n';
}
```

11.2 MPI-distributed GMRES

Based on tests/test_mpi.cpp. Requires -DOWT_KRYLOV_ENABLE_MPI=ON; run with 2 ranks.

```
#include <owt/krylov/owt_krylov.hpp>
#include <mpi.h>
#include <iostream>

int main(int argc, char** argv) {
```

```

MPI_Init(&argc, &argv);
using namespace owt::krylov;
int rank = 0, size = 0;
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);    // expects size == 2

using Halo = MpiHaloExchange<double>;
Halo::Neighbor neighbor;
neighbor.rank = 1 - rank;
neighbor.send_owned_nodes = {0};
neighbor.receive_ghost_nodes = {0};

BlockCsrMatrix<double> matrix(1, 1, 2, {0, 2}, {0, 1}, {2, 2, -1, -1});
const JacobiPreconditioner jacobi(matrix);
Halo halo(MPI_COMM_WORLD, 1, 1, 2, {neighbor});
OverlappedDistributedBlockOperator distributed_operator(matrix, std::move(halo));

BlockVector<double> exact(1, 1, 2);
exact.node(0)[0] = double(rank + 1);
exact.node(0)[1] = double(2 * (rank + 1));
BlockVector<double> rhs = exact.clone_layout();
BlockVector<double> exact_copy = exact;
distributed_operator.apply(exact_copy, rhs);    // fills rhs, exchanges exact's
ghost

BlockVector<double> solution = exact.clone_layout();
SolverOptions<double> options;
options.relative_tolerance = 1e-12;
options.convergence_check_interval = 1;
options.maximum_iterations = 50;
options.restart = 8;
const auto result = gmres(distributed_operator, rhs, solution, options,
                          jacobi, MpiReduction<double>(MPI_COMM_WORLD));
if (rank == 0) std::cout << "converged=" << result.converged() << '\n';
MPI_Finalize();
}

```

11.3 Restricted Additive Schwarz (depth-1)

Based on `tests/test_mpi.cpp`, continuing §11.2's setup (communicator, matrix, rhs, exact, distributed_operator).

```

DistributedLayout overlap_layout(
    /*global_nodes=*/2, /*block_size=*/2,
    /*owned_global=*/{std::uint64_t(rank)}, /*ghost_global=*/{std::uint64_t(1 - rank)},
    /*ghost_owners=*/{1 - rank},
    /*algebraic=*/{std::uint64_t(rank), std::uint64_t(1 - rank)});

auto overlap = build_depth_one_overlap(MPI_COMM_WORLD, overlap_layout, matrix);

MpiHaloExchange<double>::Neighbor ras_neighbor;
ras_neighbor.rank = 1 - rank;
ras_neighbor.send_owned_nodes = {0};
ras_neighbor.receive_ghost_nodes = {0};
MpiHaloExchange<double> ras_halo(MPI_COMM_WORLD, 1, 1, 2, {ras_neighbor}, /*tag=*/
9175);

```

```

RestrictedAdditiveSchwarzIlu0<double, std::size_t, MPIHaloExchange<double>> ras(
    rhs, std::move(overlap.matrix), std::move(overlap.outer_local_nodes),
    ras_halo, /*overlap_depth=*/1);

SolverOptions<double> options;
options.relative_tolerance = 1e-12;
options.convergence_check_interval = 1;
options.maximum_iterations = 50;
options.restart = 8;

BlockVector<double> ras_solution = exact.clone_layout();
const auto ras_result = gmres(distributed_operator, rhs, ras_solution, options,
                             ras, MPIReduction<double>(MPI_COMM_WORLD));
// exact depth-one overlap converges the exact local correction in <= 1 GMRES
iteration

```

For a many-rank, cached, arbitrary-depth variant, see DistributedOverlapPlan + CachedRestrictedAdditiveSchwarz in tests/test_mpi_partitions.cpp.

11.4 PETSc adapter with independent residual check

Based on tests/test_mpi.cpp. Requires -DOWT_KRYLOV_ENABLE_MPI=ON -DOWT_KRYLOV_ENABLE_PETSC=ON.

```

#include <owt/krylov/owt_krylov.hpp>
#include <mpi.h>
#include <petscksp.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    PetscInitialize(&argc, &argv, nullptr, nullptr);
    using namespace owt::krylov;
    int rank = 0;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    BlockCsrMatrix<double> matrix(1, 1, 2, {0, 2}, {0, 1}, {2, 2, -1, -1});
    DistributedLayout layout(2, 2, {std::uint64_t(rank)}, {std::uint64_t(1 - rank)},
                             {1 - rank}, {std::uint64_t(rank), std::uint64_t(1 - rank)});

    PetscSolverOptions petsc_options;
    petsc_options.ksp_type = KSPBCGS;
    petsc_options.pc_type = PCJACOBI;
    petsc_options.relative_tolerance = 1e-12;
    PetscBlockCsrSolver solver(MPI_COMM_WORLD, layout, matrix, petsc_options);

    BlockVector<double> rhs(1, 1, 2);
    rhs.node(0)[0] = rank == 0 ? 0.0 : 3.0;
    rhs.node(0)[1] = rank == 0 ? 0.0 : 6.0;
    BlockVector<double> solution = rhs.clone_layout();
    const auto result = solver.solve(rhs, solution);
    // result.true_residual_norm is recomputed independently via MatMult, not
    // taken from KSPGetResidualNorm alone.

    PetscFinalize();
    MPI_Finalize();
}

```

```
}

```

11.5 GCRO-DR recycling across related solves

Based on `tests/test_core.cpp`. Requires `-DWT_KRYLOV_ENABLE_LAPACK=ON`; continues §11.1's setup.

```
RecycleSpace<double> recycle_space(/*maximum_vectors=*/2);
ArnoldiSnapshot<double> snapshot;

BlockVector<double> first_solution = exact.clone_layout();
const auto first = gcrodr(operator_view, rhs, first_solution, recycle_space, options,
                          jacobi, SerialReduction<double>{},
                          static_cast<SolverWorkspace<double>*>(nullptr), &snapshot);
// recycle_space now holds up to 2 harmonic-Ritz vectors

// Solve a related (perturbed) system, reusing the recycled subspace.
BlockVector<double> perturbed_exact = exact;
perturbed_exact.node(0)[0] += 0.25;
perturbed_exact.node(2)[1] -= 0.5;
BlockVector<double> perturbed_rhs = exact.clone_layout();
BlockVector<double> perturbed_copy = perturbed_exact;
operator_view.apply(perturbed_copy, perturbed_rhs);

BlockVector<double> second_solution = exact.clone_layout();
const auto second = gcrodr(operator_view, perturbed_rhs, second_solution,
                          recycle_space, options, jacobi);
// second converges faster because recycle_space was seeded by the first solve
```

A simpler `recycled_fgmres` example (same idea, no harmonic-Ritz extraction, no `WT_KRYLOV_ENABLE_LAPACK` needed) is in `tests/test_core.cpp` as well.

12 Benchmarking

12.1 Unstructured development benchmark

```
cmake -S . -B build-bench -DOWT_KRYLOV_BUILD_BENCHMARKS=ON
cmake --build build-bench
./build-bench/benchmarks/owt_krylov_unstructured_benchmark 64 32
```

Emits CSV with iterations, applications, reductions, elapsed solve time, and verified residual. This is a regression/engineering fixture, not a substitute for the SpecWave Limon comparison below.

With MPI enabled, `owt_krylov_distributed_benchmark` constructs one slab-partitioned problem and runs native OWT and (when enabled) PETSc on the same matrix, partition, initial guess, unpreconditioned stopping norm, and verified residual — the PETSc adapter uses exact diagonal/off-diagonal AIJ preallocation, and its structural setup and numeric update are reported separately from solve time.

12.2 SpecWave Limon protocol

`benchmarks/specwave_limon.sh` runs the native SpecWave baseline, the borrowed OWT-Krylov adapter, and PETSc from **independent temporary copies** of the same Limon case. It does not edit `wwx_bench.nml`, `system.dat`, reference results, or manuscript/observational data; SpecWave's `--solver` CLI flag only changes the in-memory configuration for that run.

Controlled fields: one `system.dat`, NML config, partitioner, rank count, spectral block (36×36), initial wave field, time-step count, stopping tolerance; one executable, compiler optimization set, MPI implementation, and host; independently computed $\|b - Ax\|_2 / \|b\|_2$ for both OWT-Krylov and PETSc; separate structural-setup, numeric-update/assembly, and solve telemetry where the backend exposes it.

```
make -C ../SpecWave/TRITON-C/ww-x -f Makefile.linux \
  USE_OWT_KRYLOV=1 USE_PETSC=1
WWX=/path/to/ww-x MPI_RANKS=4 benchmarks/specwave_limon.sh
```

The script records repository revisions, host, MPI launcher, rank count, and a CSV row per solver. A timing comparison is valid only when exit codes are zero *and* independently verified residuals satisfy the same tolerance across variants. **Known limitation:** legacy native ID 12 (`pipelined_full_system`) currently has application convergence output but no independent linear true-residual hook connected, so it remains a performance baseline only — not a strict residual-matched comparison — until that monitor is connected.

13 Testing

```
ctest --test-dir build --output-on-failure          # serial suite
ctest --test-dir build-mpi --output-on-failure      # MPI suite (needs
OWT_KRYLOV_ENABLE_MPI)
```

Test binaries (`tests/CMakeLists.txt`):

Binary	Ranks	Covers
owt_krylov_core_tests	1 (serial)	Every native SpecWave solver ID, GMRES/BiCGSTAB/IDR(s)/GCRO-DR, multigrid, ILU(0), local SSOR
owt_krylov_mpi_tests	2	Owned/ghost reduction rule, direct block halo exchange, automatic overlap import, RAS, distributed coarse correction, PETSc adapter
owt_krylov_partition_tests	2 and 4	Partitioned solves, cached arbitrary-depth RAS via <code>DistributedOverlapPlan</code>

The serial suite exercises every native SpecWave solver ID 0–21. The MPI suite verifies the owned/ghost reduction rule, direct block halo exchange, automatic overlap import, RAS, the distributed coarse correction, and partitioned solves; with PETSc enabled, the two-rank test also assembles and solves through the PETSc adapter.

14 Known rough edges

Findings from a direct line-by-line reading of the headers, worth knowing before relying on the affected behavior:

- **ordering.hpp's NodeOrdering enum is never consumed anywhere in the library.** It is a labeling vocabulary only; there is no `order(matrix, NodeOrdering::x)` dispatcher and no solver/preconditioner applies a reordering automatically (§4.3).
- **approximate_minimum_degree_order is a plain $O(n^2)$ greedy minimum-degree ordering,** not the near-linear AMD algorithm the name suggests — no quotient-graph or approximate-degree optimization (§4.3).
- **RasSsorPreconditioner is a compatibility alias for LocalSsorPreconditioner,** not an actual overlapping RAS preconditioner — the class's own doc comment says so explicitly (§6.2). Use `RestrictedAdditiveSchwarzIlu0/CachedRestrictedAdditiveSchwarzIlu0` for genuine RAS.
- **solver_type=22/OWTKrylovSpecWaveSolver do not exist in this repository** — they live in the sibling SpecWave application repository, which consumes OWT-Krylov's types by reference (§10.1).
- **solve_specwave_native dispatches legacy IDs 5, 8, and 11 identically** (all three call `bicgstab` with `LocalSsorPreconditioner`), even though `docs/SPECWAVE_PORT_MATRIX.md` describes ID 11 without mentioning SSOR (§10.2).
- **SolverOptions::relaxation is inert for every Krylov solver in krylov_solvers.hpp/pipelined_bicg** — it is read only by `stationary.hpp's jacobi()` (§5.2). `SolverOptions::verify_true_residual` is not branched on anywhere in the reviewed solvers — true-residual verification always happens.
- **ids() ignores options.residual_replacement_interval** entirely, and has no `SolverWorkspace`, `*_stable`, `*_mixed_precision`, or `*_residual_replacement` overloads unlike every BiCGSTAB variant (§5.6).
- **pipelined_bicgstab_residual_replacement/communication_hiding_bicgstab_residual_replacement** **hardcode SerialReduction<T>**, silently ignoring any `Reduction` the caller otherwise uses — unlike the plain-bicgstab residual-replacement wrapper, which forwards the caller's choice (§5.5).
- **DistributedLayout never cross-validates algebraic_local_to_global_nodes() against owned_global_nodes()/ghost_global_nodes()** — it checks each array's internal consistency independently, so a caller could construct a layout where the two are mutually inconsistent without triggering an exception (§3.3).
- **overlap.hpp/overlap_plan.hpp have an inconsistent collective-safety guarantee for missing-row errors:** `DistributedOverlapPlan::import_layer` agrees a missing-row failure across all ranks via `MPI_Allreduce` before throwing, but `build_depth_one_overlap` and `DistributedOverlapPlan::build_residual_schedule` throw locally with no cross-rank agreement — a request naming an unowned node could make one rank throw while others proceed into a later collective (§6.4).
- **SplitLocalSsorPreconditioner's internal as_size() has no negative-value guard,** unlike the equivalent helper in every other matrix type in the library (`BlockCsrMatrix`, `DenseBlockCsrMatrix`, `SplitBlockCsrMatrixView` all throw on a negative signed `Index`) (§6.2).

- **JacobiPreconditioner's CTAD deduction guide's default tolerance ($\tau(0)$) doesn't match the real constructor's default ($64 \cdot \text{eps}$)** — purely cosmetic (deduction guides only steer template-argument deduction), but worth knowing if you read the guide as documentation (§6.1).
- **OpenMPTargetExecutionPolicy re-transfers the entire matrix and vectors to the device on every `apply()` call** — no persistent device-resident state exists in this header (§4.2).

15 Claim boundaries

Carried forward verbatim in spirit from `README.md` and `docs/ROADMAP.md` — read these before citing a performance number from this library:

- No OWT-versus-PETSc speed claim without the same matrix, partition, initial guess, tolerance, and an independently verified $\|b - Ax\|$.
- No scalable-coarse claim from `SubdomainConstantCoarseCorrection` alone — it is a replicated dense small-rank baseline; `PetscSubdomainCoarseCorrection` is the scale-appropriate choice.
- No reproducibility claim across different partitions from `MpiDeterministicReduction` — it is deterministic only for a fixed partition and rank ordering, not partition-independent.
- No GCRO-DR claim without harmonic-Ritz extraction and a corresponding repeated-system test (both exist and are exercised, but the claim is scoped to that configuration).
- OpenMP Target is an implemented portability backend; no GPU speed claim is made without device-resident workload measurements (see the transfer-cost note in §4.2 and §14).
- The SpecWave adapter and non-mutating Limon protocol are implemented, but OWT makes no new application-level native-over-PETSc timing claim until a recorded run has zero exit codes and matched independently verified residuals (§12.2).

16 Header index

All under `include/owt/krylov/`. `owt_krylov.hpp` includes every header in this table.

Header	Provides	Build flag
<code>core.hpp</code>	<code>BlockVector</code> , <code>SolverOptions</code> , <code>SolverResult</code> , <code>SolverWorkspace</code> , <code>ArnoldiSnapshot</code> , BLAS-1 free functions	—
<code>block_csr.hpp</code>	<code>BlockCsrMatrix</code> , <code>NoHaloExchange</code> , <code>DistributedBlockOperator</code>	—
<code>dense_block_csr.hpp</code>	<code>DenseBlockCsrMatrix</code> (true dense block coupling)	—
<code>split_block_csr.hpp</code>	<code>SplitBlockCsrMatrixView</code> , <code>SplitLocalSsorPreconditioner</code>	—
<code>distributed_layout.hpp</code>	<code>DistributedLayout</code>	—
<code>execution.hpp</code>	<code>HostExecutionPolicy</code> , <code>OpenMPTargetExecutionPolicy</code> , <code>PolicyBlockOperator</code>	<code>OWT_KRYLOV_ENABLE</code> <code>_OPENMP_TARGET</code> gates the target policy
<code>simd.hpp</code>	<code>simd::fused_multiply_add</code> , <code>simd::axpy</code>	— (ISA macros, not CMake options)
<code>ordering.hpp</code>	RCM, AMD-style, nested-dissection, Hilbert node orderings	—
<code>reduction.hpp</code>	<code>SerialReduction</code> , <code>MixedPrecisionSerialReduction</code> , <code>MpiReduction</code> , <code>MpiDeterministicReduction</code> , <code>MpiMixedPrecisionReduction</code>	MPI types gated on <code>OWT</code> <code>_KRYLOV_ENABLE_MPI</code>
<code>krylov_solvers.hpp</code>	<code>gmres/fgmres</code> , <code>bicgstab</code> + policy wrappers	—
<code>pipelined_bicgstab.hpp</code>	<code>pipelined_bicgstab</code> , <code>communication_hiding_bicgstab</code>	—
<code>idrs.hpp</code>	<code>idrs</code>	—
<code>recycling.hpp</code>	<code>RecycleSpace</code> , <code>recycled_fgmres</code>	—
<code>gcrodr.hpp</code>	<code>extract_harmonic_ritz</code> , <code>gcrodr</code>	<code>OWT_KRYLOV_ENABLE</code> <code>_LAPACK</code>
<code>preconditioner.hpp</code>	<code>IdentityPreconditioner</code> , <code>JacobiPreconditioner</code> , <code>LocalSsorPreconditioner</code> , <code>Ilu0Preconditioner</code> , <code>RestrictedAdditiveSchwarzIlu0</code>	—
<code>stationary.hpp</code>	<code>jacobi</code> , <code>chebyshev_srj</code> , <code>gauss_seidel</code> , <code>asynchronous_gauss_seidel</code> , <code>anderson_jacobi</code>	—
<code>overlap.hpp</code>	<code>build_depth_one_overlap</code>	<code>OWT_KRYLOV_ENABLE_MPI</code>
<code>overlap_plan.hpp</code>	<code>DistributedOverlapPlan</code> , <code>CachedRestrictedAdditiveSchwarzIlu0</code>	<code>OWT_KRYLOV_ENABLE_MPI</code>

Header	Provides	Build flag
<code>coarse.hpp</code>	<code>SubdomainConstantCoarseCorrection</code> , <code>MultiplicativeTwoLevelPreconditioner</code>	<code>OWT_KRYLOV_ENABLE_MPI</code>
<code>multigrid.hpp</code>	<code>AggregationMultigrid</code>	—
<code>mpi_halo.hpp</code>	<code>MpiHaloExchange</code> , <code>OverlappedDistributedBlockOperator</code>	<code>OWT_KRYLOV_ENABLE_MPI</code>
<code>petsc_adapter.hpp</code>	<code>PetscBlockCsrSolver</code>	<code>OWT_KRYLOV_ENABLE</code> <code>_PETSC</code>
<code>petsc_coarse.hpp</code>	<code>PetscSubdomainCoarseCorrection</code>	<code>OWT_KRYLOV_ENABLE</code> <code>_PETSC</code>
<code>specwave_compat.hpp</code>	<code>SpecWaveSolver</code> , <code>solve_specwave_native</code>	—
<code>owt_krylov.hpp</code>	Aggregate include of everything above	—